CS-GY 6513 | BIG DATA

Efficient Semantic Joins with Hybrid Value Matching

Project 4: Semantic Joins and Value Matching Operators

Step 2: Project Proposal
October 7, 2026

Team name: Game of Joins

Project members and NYU netIDs

Ella Bourassa - emb9769
Rajdeep Dey - rd3871
Jingrao Chen - jc15325

1. Problem and proposed approach

We will build a semantic join operator that identifies values referring to the same entity or concept across two columns, even when their text differs. Exact joins miss pairs such as “USA” and “United States of America,” while string similarity alone can confuse related but distinct values. Our goal is to improve recall over exact matching while maintaining high precision and practical execution time for columns containing thousands of unique values.

Our central question is whether a staged combination of inexpensive string methods, semantic candidate retrieval, and selective LLM verification can improve the quality-cost tradeoff. We will evaluate this hypothesis rather than assume that adding an LLM always improves results.

Stage 1: Profile and prepare the values

Using Python and PySpark, we will read the input tables, remove nulls from matching, and extract distinct values while preserving their original text and row identifiers. We will create normalized copies using conservative operations such as trimming whitespace and standardizing case. More aggressive transformations, such as punctuation removal, will be evaluated separately because they can merge different entities. Raw exact matches will provide the first baseline.

Stage 2: Retrieve a manageable set of candidate pairs

We will combine candidates from normalized equality, shared tokens or character n-grams, and nearest-neighbor retrieval over pretrained text embeddings. Embeddings will be computed once per distinct value and reused. Retrieval in both directions will help recover pairs that a one-sided search misses. We will tune the number of neighbors on development data and measure candidate recall before verification. Embedding similarity will identify candidates, not establish semantic equivalence.

Stage 3: Score and verify candidates

We will score candidates with edit-distance and token-based similarity. Thresholds chosen on development data will determine which pairs are accepted, rejected, or considered uncertain. If model access and budget permit, an LLM will evaluate uncertain pairs using the original values and available column context, returning match, non-match, or uncertain. Small batches and cached decisions will limit repeated work, drawing on the batching direction in [5]. An uncertain response will remain unresolved and will not be silently accepted. The system will also run with LLM verification disabled.

Stage 4: Produce the semantic join

The operator will output a table of accepted value pairs with the method responsible for each decision and any similarity scores. This mapping will be joined back to the original tables to preserve duplicate rows and valid many-to-many matches. We will not enforce one-to-one matching. A documented interface will accept two tables, their join columns, and a configuration for retrieval, thresholds, and the optional verifier.

2. Implementation and evaluation

Course concepts and scope

PySpark DataFrames will support distinct-value extraction, candidate blocking, deduplication, and joining the accepted mapping back to the source tables. We will inspect execution plans and measure the effects of partitioning and small-table broadcast joins where appropriate. Embedding retrieval will initially use a local index over distinct values; its overhead and memory use will be reported separately. We will avoid a full Cartesian product in the main pipeline, although blocking can still generate many pairs when a block is large.

The core deliverable is a working hybrid operator and reproducible evaluation. LLM verification is a budget-dependent extension, and integration with BDIKit is a stretch goal. We will focus on value equivalence, rather than arbitrary natural-language join predicates or full multi-attribute entity resolution.

Data and experimental design

Our primary evaluation source will be the Auto-FuzzyJoin benchmark identified in the project description [2]. We will first inspect its files and ground-truth conventions, then select column pairs covering different domains and discrepancy types. We will separate development and held-out test column pairs before tuning; related tables will remain in the same split where feasible. Any examples used in prompts will come only from development data. Test labels will be reserved for final evaluation.

We will compare raw exact joins, normalized exact joins, string-similarity matching, embedding-based matching, and the combined pipeline. Ablations will remove semantic retrieval and LLM verification individually. All methods will use the same evaluation splits. If time permits, Freyja will provide an additional qualitative case study; without complete value-pair labels, sampled manual review can estimate precision but cannot establish full recall.

Measures of effectiveness and efficiency

Match quality: precision, recall, and F1 over unique value pairs, plus candidate recall before scoring. We will report results per column pair and across datasets, and analyze errors involving typos, formatting, abbreviations, aliases, and ambiguous names. This will distinguish retrieval failures from verification failures.

Cost and scale: end-to-end runtime, stage-level runtime, peak memory where measurable, candidate count, and LLM calls and tokens. We will report API cost if applicable, hardware, model versions, and cache state. Controlled scale experiments will target 100, 1,000, and 5,000 distinct values per column where data permits; any synthetic enlargement will be labeled and kept separate from benchmark accuracy results. Repeated timing runs will be summarized with medians.

Success criteria and risks

We aim to improve held-out recall over exact joins and F1 over the strongest individual baseline, while reducing candidate comparisons relative to all-pairs matching. These are experimental targets, not

promised results. Main risks are missed candidates, misleading embedding similarity, ambiguous abbreviations, and unreliable LLM decisions. We will vary retrieval limits, examine difficult negatives, retain unresolved cases, and report limitations. If LLM access is unavailable, the non-LLM pipeline and its evaluation will still constitute a complete project.

3. Milestones and deliverables

We will follow the milestones below to complete implementation, evaluation, and documentation before the final report and presentation. Intermediate dates are internal team targets. We will meet weekly to review progress, address blockers, and track each member's contributions. All dates are in 2026.

Target date	Milestone and completion evidence
Oct 7	Project proposal: submit the approach, evaluation plan, and milestones.
Oct 8–14	Proposal feedback: incorporate instructor feedback, refine the scope, and assign responsibilities.
Oct 14	Data and setup: inspect benchmark labels, define development/test splits, and create the repository and data loader.
Oct 21	Baselines: implement exact, normalized, and string-similarity joins; record precision, recall, and F1.
Oct 28	Candidate retrieval: add blocking and embedding search; measure candidate recall and comparison counts.
Nov 4	End-to-end prototype: combine retrieval and scoring, produce value mappings, and join them back to source rows in PySpark.
Nov 11	Optional LLM verification: add batching and caching if feasible; otherwise refine the non-LLM pipeline. Freeze configurations using development data.
Nov 18	Experiments: run held-out comparisons, ablations, and scale tests; record match quality, runtime, memory, and model usage.
Nov 25	Analysis and report draft: examine errors and tradeoffs, prepare results tables and figures, and draft methods and results.
Dec 2	Complete draft and reproducibility check: finish the report and repository documentation, reproduce the results, and prepare slides.
Dec 7	Final review and rehearsal: verify references and repository links, document member contributions, and rehearse the seven-minute presentation and Q&A.
Dec 9	Final submissions: submit the final report. Submit presentation slides as a PDF through Brightspace by 5 p.m.
Dec 10	Presentation: deliver the seven-minute presentation followed by five minutes of Q&A. All members will attend and participate in answering questions.

Final outputs

A code repository with setup instructions, a documented join interface, examples, experiment configurations, and reproducible evaluation scripts; a research-paper-style report covering the problem, related work, methods, results, limitations, references, repository link, and each member's contributions;

and a seven-minute presentation with the project title, team members, repository link, key findings, and limitations.

References and evaluation resources

[1] Zhu, E., He, Y., and Chaudhuri, S. (2017). Auto-join: Joining tables by leveraging transformations. PVLDB 10(10), 1034-1045.
[2] Li, P., et al. (2021). Auto-fuzzyjoin: Auto-program fuzzy similarity joins without labeled examples. SIGMOD. Benchmark: AutomaticFuzzyJoin repository.
[3] Xu, P., and Lu, J. (2019). Towards a unified framework for string similarity joins. PVLDB 12(11), 1289-1302.
[4] Dargahi Nobari, A., and Rafiei, D. (2024). Dtt: An example-driven tabular transformer for joinability by leveraging large language models. PACMMOD 2(1), 1-24.
[5] Trummer, I. (2025). Implementing Semantic Join Operators Efficiently. arXiv:2510.08489.